# Spark Memory Management Deep Dive

## Prepared By: Kaviraj T U

## Heap vs Off-Heap | Execution vs Storage (What Really Matters)

👉 If your Spark jobs fail with OOM or spill endlessly, the problem is rarely “not enough memory.”

👉 It’s usually **how memory is managed**.

---

## 1. The Big Picture

👉 Spark memory is divided into two dimensions:

👉 **Where it lives**

- Heap (JVM memory)
- Off-Heap (native memory)

👉 **How it’s used**

- Execution memory
- Storage memory

👉 Understanding this = **predictable performance**

---

## 2. Heap Memory (Default Zone)

👉 Managed by JVM (Garbage Collection)

👉 Stores:

- Objects (RDDs, DataFrames)
- Shuffle buffers
- Execution data

👉 Challenges:

- GC pauses
- Fragmentation
- OOM errors

💡 Insight:

✔️ Too many objects = GC overhead > compute time

---

## 3. Off-Heap Memory (The Hidden Weapon)

👉 Enabled via:

```
spark.memory.offHeap.enabled=true
spark.memory.offHeap.size=...
```

👉 Managed outside JVM

👉 Used by Tungsten engine

👉 Stores:

- Serialized data
- Unsafe rows

👉 Benefits:

- Less GC pressure
- Faster memory access
- Better CPU efficiency

👉 Risk:

- Not visible to JVM → can crash container if misconfigured

---

## 4. Execution Memory (For Computation)

👉 Used for:

- Shuffles
- Joins
- Aggregations
- Sorts

👉 Dynamic allocation:

- Can borrow from storage if needed

💡 Insight:

✔️ Insufficient execution memory = **spills to disk**

---

## 5. Storage Memory (For Caching)

👉 Used for:

- Cached DataFrames/RDDs
- Broadcast variables

👉 Behavior:

- Evicts old data if space needed
- Cannot evict execution memory

💡 Insight:

✔️ Over-caching = starving execution memory

---

## 6. Unified Memory Model (Important!)

👉 Spark uses a **shared pool**:

- Execution + Storage share memory
- Execution can evict storage
- Storage cannot evict execution

👉 This balance is key to stability

---

## 7. Spill = Performance Warning

👉 When memory is insufficient:

- Data spills to disk
- Jobs slow down drastically

👉 Watch for:

- Disk Spill
- Memory Spill

💡 If you see spills → tuning needed immediately

---

## 8. Real Production Tuning Tips

✔️ Increase executor memory *strategically*
✔️ Tune `spark.sql.shuffle.partitions`
✔️ Avoid over-caching
✔️ Use off-heap for heavy workloads
✔️ Prefer serialized storage (`MEMORY_AND_DISK_SER`)

---

## 9. Common Anti-Patterns

❌ Blindly increasing memory

❌ Caching everything

❌ Ignoring GC logs

❌ Not using Kryo serialization

---

## Final Takeaway

👉 **Memory is not just capacity — it's a strategy.**

- Heap → flexible but GC-heavy
- Off-heap → fast but needs control
- Execution → powers compute
- Storage → powers reuse

👉 Master this balance, and Spark stops failing...and starts scaling.

---

## Thank you.!

## Happy Learning...